ES234317 - Data Structure and Algorithms

Ganjil | 2026-2027

Labwork 03

Module 03: Map, Set, and List

Language: Java

Type: Pre-lab

Mode: individual practical work

Learning outcomes

By completing this module, you should be able to:

1. Use List to store and manage an ordered collection of data.
2. Use Set to store and manage unique data without duplicates.
3. Use Map to store and retrieve data using key-value pairs.
4. Use common operations such as add(), get(), put(), remove(), and contains().
5. Choose the appropriate data structure based on how the data needs to be stored and accessed.
6. Explain how List, Set, and Map can be used to solve different parts of a problem.

Concept Toolkit

Concept	Meaning and a useful distinction
List	A collection that stores elements in a specific order. It allows duplicate elements and elements can be accessed using their index.
Set	A collection that stores only unique elements. Unlike a List, it does not allow duplicate elements.
Map	A collection that stores data as key-value pairs. Each key is unique and is used to access its corresponding value.
add()	Adds an element to a List or Set. If the element already exists in a Set, it will not be added again.
get()	Retrieves an element from a List using its index or retrieves a value from a Map using its key.
put()	Adds a key-value pair to a Map. If the key already exists, its value is replaced.
remove()	Removes an element from a List or Set, or removes a key-value pair from a Map.
contains()	Checks whether a List or Set contains a specific element.
containsKey()	Checks whether a specific key exists in a Map.
size()	Returns the number of elements in a List or Set, or the number of key-value pairs in a Map.

ES234317 - Data Structure and Algorithms

Ganjil | 2026-2027

Labwork 03

Unguided Case: Campus Course Enrollment Monitor

The academic office needs a program to monitor the number of students enrolled in several courses. Each course is identified by a unique course code, such as `DSA101` or `DB202`. The office receives a text file named `enrollment.txt` containing enrollment operations in the order they occur.

A course may appear in many operations. The program must keep the current enrollment count for each course, update the correct course when students register or withdraw, and retrieve the current count when the office requests it. A course code that has not appeared before must be handled appropriately depending on the operation. Repeated course codes must refer to the same course record rather than creating separate records.

Your task is to design and implement a Java program that processes the file and reports the final enrollment information and any rejected operations.

ES234317 - Data Structure and Algorithms

Ganjil | 2026-2027

Labwork 03

Unguided Tasks

A1. Implementation and demonstration

Note: Only use the methods we have already learned. Do not use other built-in methods that have not been covered in this module.

Create a Java program in a single `Main.java` file.

1. Read all operations from `enrollment.txt` using `Scanner` and process them in their original order.
2. Each line contains an operation followed by a course code and, when required, a number of students.
    - REGISTER <COURSE_CODE> <COUNT>
    - WITHDRAW <COURSE_CODE> <COUNT>
    - CHECK <COURSE_CODE>
4. Process each operation according to the following rules:
    - REGISTER: If the course has not appeared before, create a record with the given enrollment count. Otherwise, add the given count to its current enrollment.
    - WITHDRAW: If the course exists and its current enrollment is at least the requested count, subtract that count. Otherwise, reject the operation and leave the enrollment unchanged.
    - CHECK: Display the current enrollment for the specified course. If the course does not exist, display that it is not found.
5. If a registration or withdrawal count is zero or negative, reject the operation and count it as an invalid operation. Do not change any enrollment record.
6. After processing all lines, display:
    - The final enrollment count for each course, in the order each course first appeared in the file.
    - The total number of rejected operations.
    - The result of each `CHECK` operation in the order it appeared in the input.

A2. Required checks

Create a text file named `enrollment.txt` with the following content:

```
REGISTER DSA101 30
REGISTER DB202 25
REGISTER DSA101 5
CHECK DSA101
WITHDRAW DB202 10
CHECK DB202
WITHDRAW DB202 20
REGISTER UX110 18
CHECK UX110
WITHDRAW NET300 2
REGISTER DB202 0
CHECK NET300
```

Expected Output:

```
===== Enrollment Checks =====
DSA101: 35 students
DB202: 15 students
UX110: 18 students
NET300: Not found

===== Final Enrollment =====
DSA101: 35 students
DB202: 15 students
UX110: 18 students

Rejected operations: 3
```

Note: CHECK operations do not create a course record. The rejected operations in the sample are:

- WITHDRAW DB202 20 — insufficient enrollment.
- WITHDRAW NET300 2 — course does not exist.
- REGISTER DB202 0 — invalid enrollment count.

Submission

Please follow these instructions carefully to submit your work:

1. Create a folder named `unguided` inside your `lw03` folder.
2. Save `Main.java` and `enrollment.txt` inside the `unguided` folder.
3. Run the program and verify that its output matches the expected output.
4. Commit and push your work to GitHub with the commit message:
   `Labwork 03 – Unguided`
5. Deadline: Follow the deadline announced by your instructor.

Labwork 03


dsa-[nrp]/
└── src/
├── lw01/
│ ├── prelab/
│ │ └── [your prelab files]
│ └── unguided/
│ └── [your unguided files]
├── lw02/
│ ├── prelab/
│ │ └── [your prelab files]
│ └── unguided/
│ └── [your unguided files]
├── lw03/
│ ├── prelab/
│ │ └── [your prelab files]
│ └── unguided/
│ └── [your unguided files]
├── ...